

1.	Prerequisite	3
2.	Install KSQL DB – 60 Minutes(D)	4
4.	Workflow using KSQL - CLI – 90 Minutes(D)	12

Date : 18th Sept 2026.

<https://docs.confluent.io/current/ksql/docs/tutorials/examples.html#ksql-examples>

1. Prerequisite

JDK : 17

Confluent : 8.2.0

Kafka : 3.9.0

Registry & KSQL DB

Since ksqlDB runs natively on Apache Kafka®, you'll need to have a Kafka cluster that ksqlDB is configured to use. Use the steps to the right to install the latest release of ksqlDB.

<https://ksqldb.io/quickstart-standalone-tarball.html#quickstart-content>

Configure Kafka confluent and kafka cluster

Note: Confluent Licence expire after 30days. Ensure to extract again within that period and all confluent/ksqldb config is store in /opt/ksqldb

2. Install KSQL DB – 60 Minutes(D)

Objective: In this self-paced lab, you will configure a Kafka broker for standalone ksqlDB operations, integrate the Confluent Schema Registry, and launch the ksqlDB server and interactive CLI.

Prerequisites

Before beginning this lab, ensure your environment meets the following conditions:

- A Kafka Node is installed.
- Confluent Schema Registry is installed (required for Avro integration).

Configure and Start the Kafka Broker

First, you need to adjust the Kafka broker configurations to support the internal topics required by ksqlDB and the transaction state in a single-node environment.

Edit the server properties: Open your Kafka configuration file.

```
#vi /opt/kafka/config/server.properties
```

Or (Kraft)

```
#vi /opt/kraft/server.properties
```

Update replication settings: Ensure the following entries are set to 1 (since this is a standalone broker):

```
transaction.state.log.replication.factor=1  
transaction.state.log.min.isr=1  
offsets.topic.replication.factor=1
```

Save and exit the file.

Restart the kafka broker or start it if not done.

```
#sh startABroker.sh
```

Start Kafka Registry

ksqlDB relies on the Schema Registry for Avro integration and schema management.

Launch the registry service: Execute your startup script to bring the Schema Registry online.

```
#sh startRegistry.sh
```

Configure standalone ksqlDB

Since ksqlDB runs natively on Apache Kafka®, you'll need to have a Kafka cluster that ksqlDB is configured to use. Use the steps to the right to install the latest release of ksqlDB.

Configure ksqlDB server

Ensure your ksqlDB server has network connectivity to Kafka.

Edit the highlighted line in `/opt/confluent/etc/ksqldb/ksql-server.properties` to match your Kafka hostname and port.

Locate the Kafka section and update the `bootstrap.servers` to match your Kafka hostname and port. Add the snappy compression setting.

```
#----- Kafka -----
```

```
# The set of Kafka brokers to bootstrap Kafka cluster information from:  
bootstrap.servers=kafkao:9092
```

```
# Enable snappy compression for the Kafka producers  
compression.type=snappy
```

To enable Schema Registry Add the following line at the end of the configuration file.

```
#----- Schema Registry -----
```

```
# Uncomment and complete the following to enable KSQL's integration to the Confluent  
Schema Registry:
```

```
ksql.schema.registry.url=http://kafkao:8081
```

confluent.license.topic.replication.factor=1

Start ksqlDB's server

ksqlDB is packaged with a startup script for development use. We'll use that here.

When you're ready to run it as a service, you'll want to manage ksqlDB with something like `systemd`.

```
[Existing config in /opt/ksqldb/ksql-server.properties]
```

```
#ksql-server-start /opt/confluent/etc/ksqldb/ksql-server.properties
```

If any issue in start up because of jar.

Download and store in the following folder.

```
#cd /opt/ksqldb/share/java/ksqldb
```

```
#wget https://repo1.maven.org/maven2/io/netty/netty-all/4.1.30.Final/netty-all-4.1.30.Final.jar
```

[This License error can be safely ignore, since we are running in single Node Cluster.

```
[2026-09-18 04:22:09,505] INFO Creating topic _confluent-command with replication factor 3. At least 3 brokers must be started concurrently to complete license registration.
```

```
(io.confluent.license.LicenseStore:510)
```

```
[2026-09-18 04:22:09,507] WARN No Valid Enterprise license detected. Confluent does not provide Enterprise support for any self-managed (Confluent Platform) components without a valid Enterprise license. (io.confluent.ksql.security.license.KsqlLicenseEventLogger:41)
```

```
[2026-09-18 04:22:09,507] ERROR Can not validate license due to
(io.confluent.common.security.license.MultiLicenseBackgroundFetcher:113)
org.apache.kafka.common.errors.TimeoutException: License topic could not be created
Caused by: org.apache.kafka.common.errors.InvalidReplicationFactorException: Replication factor:
3 larger than available brokers: 1.
```

]

If you are able to successfully start the KSQL DB server, you should get the following output.

```
[2022-02-15 16:17:02,735] INFO ksqlDB API server listening on http://0.0.0.0:8088 (io.confluent.ksql.rest.se
rver.KsqlRestApplication:405)

=====
=          - - - - -          =
=      | |  _ _ _ _ _ | |  \ | _ )      =
=      | | / / _ / _ | |  | | | _ \      =
=      | | < \ \ ( | |  | | | | ) |      =
=      | | \ \ _ / _ | |  _ / | _ /      =
=          | |          =
=      The Database purpose-built      =
=      for stream processing apps      =
=====

Copyright 2017-2021 Confluent Inc.

Server 0.23.1 listening on http://0.0.0.0:8088

To access the KSQL CLI, run:
ksql http://0.0.0.0:8088

[2022-02-15 16:17:02,813] INFO Server up and running (io.confluent.ksql.rest.server.KsqlServerMain:92)
[2022-02-15 16:17:07,390] INFO Successfully submitted metrics to Confluent via secure endpoint (io.confluent
.support.metrics.submitters.ConfluentSubmitter:146)
```

Start ksqlDB's interactive CLI

ksqlDB runs as a server which clients connect to in order to issue queries.

Run this command to connect to the ksqlDB server and enter an interactive command-line interface (CLI) session. Change the port depending on your configuration.

```
#ksql http://0.0.0.0:8088
```

Or

```
#ksql http://0.0.0.0:9999
```

```
[root@kafka0 ksqldb]# /opt/ksqldb/bin/ksql http://0.0.0.0:8088
```

```
=      =
=      =
=      =
=      =
=      =
=      =
=      =
=      =
=      =
=      =
=      =
=      =
```

The Database purpose-built
for stream processing apps

Copyright 2017-2021 Confluent Inc.

CLI v0.23.1, Server v0.23.1 located at <http://0.0.0.0:8088>

Server Status: RUNNING

Having trouble? Type 'help' (case-insensitive) for a rundown of how things work!

ksql>

Verify the connection: Once the interactive ksqlDB command prompt appears, execute your first command to verify visibility into your Kafka cluster:

#show topics;

```
ksql> show topics;
```

Kafka Topic	Partitions	Partition Replicas
default_ksql_processing_log	1	1
test	1	1
topic1	2	1

Expected Result: You will see a formatted table listing the available topics. The exact topics displayed will depend on the number of topics currently existing in your Kafka system.

-----Lab Ends Here -----

4. Workflow using KSQL - CLI – 90 Minutes(D)

Following features will be demonstrated.

- Create Topics and Produce Data
- Create and produce data to the Kafka topics pageviews and users.
- Inspect Kafka Topics by Using SHOW and PRINT Statements
- Create a Stream and Table
- Write Queries

This tutorial demonstrates a simple workflow using KSQL to write streaming queries against messages in Kafka.

To get started, you must start a Kafka cluster, including ZooKeeper and a Kafka broker.

Start Schema Registry

KSQL will then query messages from this Kafka cluster.

KSQL is installed in the Confluent Platform by default.

Confluent Kafka needs to be installed for data generation.

Create Topics and Produce Data

Create and produce data to the Kafka topics `pageviews` and `users`. These steps use the KSQL datagen that is included with Confluent Platform.

1. Create the `pageviews` topic and produce data using the data generator. The following example continuously generates data with a value in DELIMITED format.

Open a new terminal and execute the following commands:

```
kafka-topics --bootstrap-server kafka0:9092 --create --topic pageviews
```

```
ksql-datagen bootstrap-server=kafka0:9092 quickstart=pageviews format=json topic=pageviews  
maxInterval=2500
```

```
(base) [root@tos ~]#
(base) [root@tos ~]#
(base) [root@tos ~]# ksql-datagen quickstart=pageviews format=delimited topic=pa
geviews maxInterval=500
[2019-07-31 21:35:34,823] INFO AvroDataConfig values:
    schemas.cache.config = 1
    enhanced.avro.schema.support = false
    connect.meta.data = true
(io.confluent.connect.avro.AvroDataConfig:179)
1 --> ([ 1564589135082 | 'User_3' | 'Page_97' ]) ts:1564589135333
11 --> ([ 1564589135590 | 'User_7' | 'Page_66' ]) ts:1564589135591
21 --> ([ 1564589135857 | 'User_1' | 'Page_34' ]) ts:1564589135861
31 --> ([ 1564589135959 | 'User_6' | 'Page_37' ]) ts:1564589135959
41 --> ([ 1564589136036 | 'User_6' | 'Page_66' ]) ts:1564589136036
51 --> ([ 1564589136428 | 'User_2' | 'Page_98' ]) ts:1564589136428
61 --> ([ 1564589136761 | 'User_9' | 'Page_26' ]) ts:1564589136761
```

Keep the terminal open.

2. Open a new terminal
3. Produce Kafka data to the `users` topic using the data generator. The following example continuously generates data with a value in JSON format.

```
kafka-topics --bootstrap-server kafkao:9092 --create --topic users
```

```
ksql-datagen bootstrap-server=kafkao:9092 quickstart=users format=json topic=users
maxInterval=2500
```

Keep the terminal open.

```
(base) [root@tos ~]#
(base) [root@tos ~]# ksql-datagen quickstart=pageviews format=delimited topic=pa
geviews maxInterval=500
[2019-07-31 21:35:34,823] INFO AvroDataConfig values:
    schemas.cache.config = 1
    enhanced.avro.schema.support = false
    connect.meta.data = true
(io.confluent.connect.avro.AvroDataConfig:179)
1 --> ([ 1564589135082 | 'User_3' | 'Page_97' ]) ts:1564589135333
11 --> ([ 1564589135590 | 'User_7' | 'Page_66' ]) ts:1564589135591
21 --> ([ 1564589135857 | 'User_1' | 'Page_34' ]) ts:1564589135861
31 --> ([ 1564589135959 | 'User_6' | 'Page_37' ]) ts:1564589135959
41 --> ([ 1564589136036 | 'User_6' | 'Page_66' ]) ts:1564589136036
51 --> ([ 1564589136428 | 'User_2' | 'Page_98' ]) ts:1564589136428
61 --> ([ 1564589136761 | 'User_9' | 'Page_26' ]) ts:1564589136761
```

Launch the KSQL CLI

Skip this if already done earlier.

To launch the CLI, run the following command. It will route the CLI logs to the `./ksql_logs` directory, relative to your current directory. By default, the CLI will look for a KSQL Server running at `http://localhost:8088`.

```
$ LOG_DIR=./ksql_logs ksql
```

Important

By default KSQL attempts to store its logs in a directory called `logs` that is relative to the location of the `ksql` executable. For example, if `ksql` is installed at `/usr/local/bin/ksql`, then it would attempt to store its logs in `/usr/local/logs`. If you are running `ksql` from the default Confluent Platform location, `<path-to-confluent>/bin`, you must override this default behavior by using the `LOG_DIR` variable.

After KSQL is started, your terminal should resemble this.

```
(base) [root@tos apps]# LOG_DIR=./ksql_logs ksql

=====
=                                     =
=  [//  /_____/_____\  /]          =
=  [//  (_____)_____]  /]          =
=  [< \_____)_____]  /]          =
=  [ \_____)_____]  /]          =
=                                     =
=  Streaming SQL Engine for Apache Kafka®  =
=====

Copyright 2017-2018 Confluent Inc.

CLI v5.2.2, Server v5.2.2 located at http://localhost:8088

Having trouble? Type 'help' (case-insensitive) for a rundown of how things work!

ksql> █
```

Inspect Kafka Topics By Using SHOW and PRINT Statements

KSQL enables inspecting Kafka topics and messages in real time.

- Use the SHOW TOPICS statement to list the available topics in the Kafka cluster.
- Use the PRINT statement to see a topic's messages as they arrive.

In the KSQL CLI, run the following statement:

SHOW TOPICS;

Your output should resemble as shown below, it depends on the context. You should be able to see the earlier two topics, you have created earlier i.e **pageviews** and **users**:

Kafka Topic	Registered	Partitions	Partition Replicas	Consumers	ConsumerGroups
_____	_____	_____	_____	_____	_____
_confluent-metrics	false	12	1	0	0
_schemas	false	1	1	0	0
pageviews	false	1	1	0	0
users	false	1	1	0	0
_____	_____	_____	_____	_____	_____

Inspect the **users** topic by using the PRINT statement:

PRINT 'users';

Your output should resemble:

Format:JSON

```
{ "ROWTIME":1540254230041,"ROWKEY":"User_1","registertime":1516754966866,"userid":"User_1","regionid":"Region_9","gender":"MALE"}
{ "ROWTIME":1540254230081,"ROWKEY":"User_3","registertime":1491558386780,"userid":"User_3","regionid":"Region_2","gender":"MALE"}
{ "ROWTIME":1540254230091,"ROWKEY":"User_7","registertime":1514374073235,"userid":"User_7","regionid":"Region_2","gender":"OTHER"}
^C{ "ROWTIME":1540254232442,"ROWKEY":"User_4","registertime":1510034151376,"userid":"User_4","regionid":"Region_8","gender":"FEMALE"}
Topic printing ceased
```

Press CTRL+C to stop printing messages.

Inspect the `pageviews` topic by using the PRINT statement:

```
PRINT 'pageviews';
```

Your output should resemble:

Format:STRING

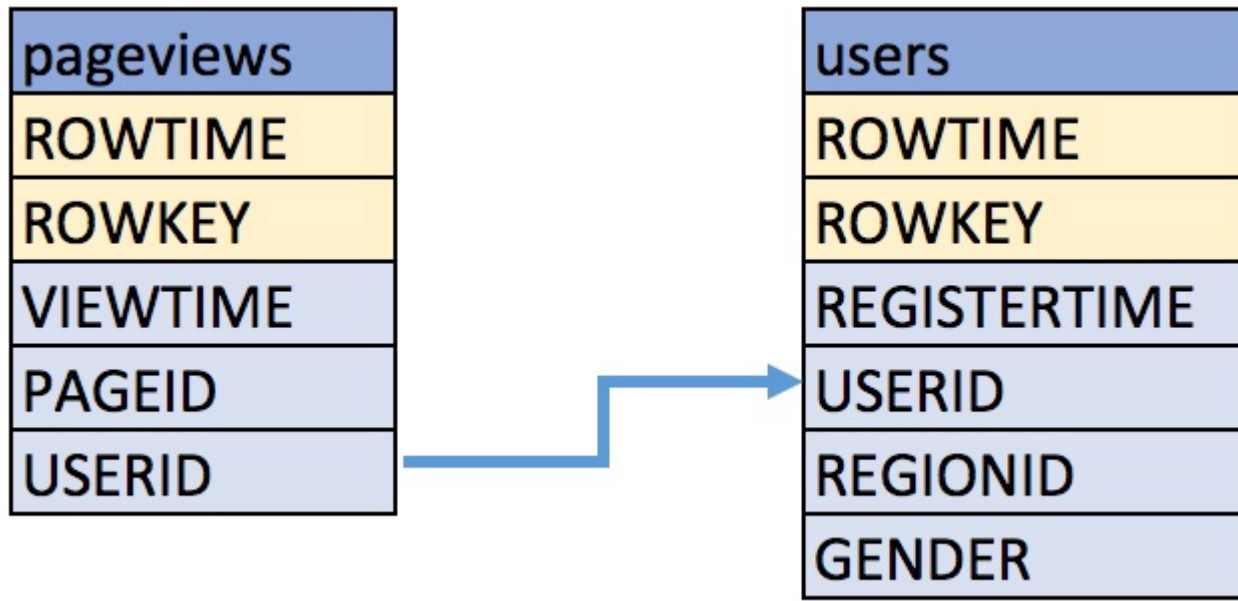
```
10/23/18 12:24:03 AM UTC , 9461 , 1540254243183,User_9,Page_20
10/23/18 12:24:03 AM UTC , 9471 , 1540254243617,User_7,Page_47
```

```
10/23/18 12:24:03 AM UTC , 9481 , 1540254243888,User_4,Page_27  
^C10/23/18 12:24:05 AM UTC , 9521 , 1540254245161,User_9,Page_62  
Topic printing ceased  
ksql>
```

Press CTRL+C to stop printing messages.

Create a Stream and Table

These examples query messages from Kafka topics called `pageviews` and `users` using the following schemas:



1. Create a stream, named `pageviews_original`, from the `pageviews` Kafka topic, specifying the `value_format` of `DELIMITED`.

```
CREATE STREAM pageviews_original (viewtime bigint, userid varchar, pageid varchar) WITH  
(kafka_topic='pageviews', value_format='JSON');
```

Your output should resemble:

```
-----  
ksql> CREATE STREAM pageviews_original (viewtime bigint, userid varchar, pageid varchar) WITH  
>(kafka_topic='pageviews', value_format='DELIMITED');  
>  
  
Message  
-----  
Stream created  
-----  
ksql> █
```

Tip

You can run `DESCRIBE pageviews_original;` to see the schema for the stream. Notice that KSQL created two additional columns, named `ROWTIME`, which corresponds with the Kafka message timestamp, and `ROWKEY`, which corresponds with the Kafka message key. – Removed in the latest version.

`DESCRIBE pageviews_original;`

```
ksql> DESCRIBE pageviews_original;

Name          : PAGEVIEWS_ORIGINAL
Field | Type
-----|-----
ROWTIME | BIGINT (system)
ROWKEY  | VARCHAR (STRING) (system)
VIEWTIME | BIGINT
USERID  | VARCHAR (STRING)
PAGEID  | VARCHAR (STRING)

For runtime statistics and query details run: DESCRIBE EXTENDED <Stream,Table>;
ksql>
```

2. Create a table, named `users_original`, from the `users` Kafka topic, specifying the `value_format` of `JSON`.

```
CREATE TABLE users_original (registertime BIGINT, gender VARCHAR, regionid VARCHAR, userid VARCHAR PRIMARY KEY) WITH (kafka_topic='users', value_format='JSON');
```

Your output should resemble:

```
Message
-----
Table created
-----
```

Tip

You can run `DESCRIBE users_original;` to see the schema for the Table.

3. Optional: Show all streams and tables.

`ksql> SHOW STREAMS;`

Stream Name	Kafka Topic	Format
PAGEVIEWS_ORIGINAL	pageviews	DELIMITED

`ksql> SHOW TABLES;`

Table Name	Kafka Topic	Format	Windowed
USERS_ORIGINAL	users	JSON	false

Write Queries

```
SET 'auto.offset.reset'='earliest';
```

These examples write queries using KSQL.

Note: By default KSQL reads the topics for streams and tables from the latest offset.

1. Use **SELECT** to create a query that returns data from a STREAM. This query includes the **LIMIT** keyword to limit the number of rows returned in the query result. Note that exact data output may vary because of the randomness of the data generation.

```
SELECT pageid FROM pageviews_original EMIT changes LIMIT 3;
```

Your output should resemble:

```
Page_24  
Page_73  
Page_78  
LIMIT reached  
Query terminated
```

2. Create a persistent query by using the `CREATE STREAM` keywords to precede the `SELECT` statement. The results from this query are written to the `PAGEVIEWS_ENRICHED` Kafka topic. The following query enriches the `pageviews_original` STREAM by doing a `LEFT JOIN` with the `users_original` TABLE on the user ID.

```
CREATE STREAM pageviews_enriched AS
SELECT users_original.userid AS userid, pageid, regionid, gender
FROM pageviews_original
JOIN users_original
  ON pageviews_original.userid = users_original.userid
Emit changes;
```

Your output should resemble:

Message

Stream created **and** running

Tip

You can run `DESCRIBE pageviews_enriched;` to describe the stream.

3. Use **SELECT** to view query results as they come in. To stop viewing the query results, press **<ctrl-c>**. This stops printing to the console but it does not terminate the actual query. The query continues to run in the underlying KSQL application.

SELECT * FROM pageviews_enriched Emit Changes;

Your output should resemble:

IUser_9	IPage_92	IRegion_2	IMALE	
IUser_2	IPage_66	IRegion_6	IMALE	
IUser_3	IPage_10	IRegion_7	IMALE	
IUser_5	IPage_30	IRegion_3	IOTHER	
IUser_2	IPage_85	IRegion_6	IMALE	
IUser_1	IPage_46	IRegion_7	IOTHER	
IUser_6	IPage_56	IRegion_3	IFEMALE	
IUser_8	IPage_13	IRegion_2	IMALE	
IUser_4	IPage_19	IRegion_4	IFEMALE	
IUser_3	IPage_44	IRegion_7	IMALE	
IUser_8	IPage_57	IRegion_2	IMALE	
IUser_8	IPage_39	IRegion_2	IMALE	
IUser_9	IPage_15	IRegion_2	IMALE	
IUser_9	IPage_71	IRegion_2	IMALE	
IUser_7	IPage_69	IRegion_8	IMALE	

4. Create a new persistent query where a condition limits the streams content, using **WHERE**. Results from this query are written to a Kafka topic called **PAGEVIEWS_FEMALE**.

```
CREATE STREAM pageviews_female AS  
SELECT * FROM pageviews_enriched  
WHERE gender = 'FEMALE';
```

Your output should resemble:

Message

Stream created **and** running

Tip

You can run `DESCRIBE pageviews_female;` to describe the stream.

5. Create a new persistent query where another condition is met, using `LIKE`. Results from this query are written to the `pageviews_enriched_r8_r9` Kafka topic.

```
CREATE STREAM pageviews_female_like_89  
WITH (kafka_topic='pageviews_enriched_r8_r9') AS  
SELECT * FROM pageviews_female  
WHERE regionid LIKE '%_8' OR regionid LIKE '%_9';
```

Your output should resemble:

Message

Stream created **and** running

6. Verify the above 2 streams:

`select * from PAGEVIEWS_FEMALE_LIKE_89 emit changes limit 6;`
`select * from PAGEVIEWS_FEMALE emit changes limit 3;`

```
ksql> select * from PAGEVIEWS_FEMALE_LIKE_89 emit changes limit 6;
```

USERID	PAGEID	REGIONID	GENDER
User_9	Page_15	Region_9	FEMALE
User_9	Page_17	Region_8	FEMALE
User_9	Page_66	Region_8	FEMALE
User_9	Page_62	Region_8	FEMALE
User_9	Page_71	Region_8	FEMALE
User_6	Page_31	Region_8	FEMALE

```
Limit Reached
Query terminated
ksql> select * from PAGEVIEWS_FEMALE emit changes limit 3;
```

USERID	PAGEID	REGIONID	GENDER
User_1	Page_30	Region_8	FEMALE
User_3	Page_23	Region_6	FEMALE
User_1	Page_81	Region_8	FEMALE

```
Limit Reached
Query terminated
ksql>
```

7. Create a new persistent query that counts the pageviews for each region combination in a **tumbling window** of 30 seconds when the count is greater than one. Results from this query are written to the **PAGEVIEWS_REGIONS** Kafka topic in the Avro format. **KSQL** will register the Avro schema with the configured Schema Registry when it writes the first message to the **PAGEVIEWS_REGIONS** topic.

```
CREATE TABLE pageviews_regions
WITH (
  KAFKA_TOPIC = 'pageviews_regions',VALUE_FORMAT='AVRO'
) AS
SELECT regionid , COUNT(*) AS numusers
FROM pageviews_enriched
  WINDOW TUMBLING (size 30 second)
GROUP BY regionid
HAVING COUNT(*) > 1 emit changes;
```

Your output should resemble:

Message

Table created **and** running

Tip

You can run `DESCRIBE pageviews_regions;` to describe the table.

8. Optional: View results from the above queries using `SELECT`.

`SELECT regionid, numusers FROM pageviews_regions emit changes LIMIT 5;`

Your output should resemble:

```
ksql> SELECT regionid, numusers FROM pageviews_regions emit changes LIMIT 5;
+-----+-----+
|REGIONID|NUMUSERS|
+-----+-----+
|Region_2|221     |
|Region_3|6169    |
|Region_5|10659   |
|Region_2|11476   |
|Region_9|2259    |
Limit Reached
Query terminated
```

9. Optional: Show all persistent queries.

`SHOW QUERIES;`

Your output should resemble:

Query ID	Kafka Topic	Query String
----------	-------------	--------------

```

-----
-----
-----
-----
CSAS_PAGEVIEWS_FEMALE_1 | PAGEVIEWS_FEMALE | CREATE STREA
M pageviews_female AS SELECT * FROM pageviews_enriched WHERE gender =
'FEMALE';
CTAS_PAGEVIEWS_REGIONS_3 | PAGEVIEWS_REGIONS | CREATE TABLE
pageviews_regions WITH (VALUE_FORMAT='avro') AS SELECT gender, region
id , COUNT(*) AS numusers FROM pageviews_enriched WINDOW TUMBLING
(size 30 second) GROUP BY gender, regionid HAVING COUNT(*) > 1;
CSAS_PAGEVIEWS_FEMALE_LIKE_89_2 | PAGEVIEWS_FEMALE_LIKE_89 | CRE
ATE STREAM pageviews_female_like_89 WITH (kafka_topic='pageviews_enriche
d_r8_r9') AS SELECT * FROM pageviews_female WHERE regionid LIKE '%_8' O
R regionid LIKE '%_9';
CSAS_PAGEVIEWS_ENRICHED_o | PAGEVIEWS_ENRICHED | CREATE STR
EAM pageviews_enriched AS SELECT users_original.userid AS userid, pageid, regio
nid, gender FROM pageviews_original LEFT JOIN users_original ON pagevie
ws_original.userid = users_original.userid;
-----
-----
-----
-----

```

For detailed information on a Query run: EXPLAIN <Query ID>;

10. Optional: Examine query run-time metrics and details. Observe that information including the target Kafka topic is available, as well as throughput figures for the messages being processed.

DESCRIBE PAGEVIEWS_REGIONS EXTENDED;

Your output should resemble:

```
Name      : PAGEVIEWS_REGIONS
Type      : TABLE
Key field  : KSQL_INTERNAL_COL_0|+|KSQL_INTERNAL_COL_1
Key format : STRING
Timestamp field : Not set - using <ROWTIME>
Value format : AVRO
Kafka topic : PAGEVIEWS_REGIONS (partitions: 4, replication: 1)
```

```
Field | Type
-----
ROWTIME | BIGINT      (system)
ROWKEY  | VARCHAR(STRING) (system)
GENDER  | VARCHAR(STRING)
REGIONID | VARCHAR(STRING)
NUMUSERS | BIGINT
-----
```

Queries that write into this TABLE

```

-----
CTAS_PAGEVIEWS_REGIONS_3 : CREATE TABLE pageviews_regions WITH (value_format='avro') AS SELECT gender, regionid, COUNT(*) AS numusers FROM pageviews_enriched WINDOW TUMBLING (size 30 second) GROUP BY gender, regionid HAVING COUNT(*) > 1;

```

For query topology **and** execution plan please run: EXPLAIN <QueryId>

Local runtime statistics

```

-----
messages-per-sec: 3.06 total-messages: 1827 last-message: 7/19/18 4:17:55 PM UTC
failed-messages: 0 failed-messages-per-sec: 0 last-failed: n/a
(Statistics of the local KSQL server interaction with the Kafka topic PAGEVIEWS_REGIONS)
ksql>

```

You can close the data generation windows/terminals.

----- Lab Ends Here -----

Any issues related to minimum config clean the zookeeper/kafka-logs and restart the services.